Proyecto final:

Análisis temático de proyectos de investigación

Procesamiento de Datos

Máster en Ingeniería de Telecomunicación

21 de octubre de 2026

1. Introducción

En este proyecto aplicaréis las técnicas de Machine Learning estudiadas durante la asignatura a un problema de clasificación de documentos de texto. En concreto, trabajaréis con los resúmenes de proyectos de investigación financiados por la Comisión Europea dentro del programa Horizonte 2020 (H2020). El objetivo será formular y resolver un problema de clasificación multiclase, comparando distintas formas de representar el texto y distintos modelos de clasificación, y analizar de forma crítica su rendimiento.

- El proyecto se realizará en equipos de 3 o 4 estudiantes.
- El proyecto tiene una puntuación máxima de 3 puntos, desglosados en:
  - Proyecto básico: 2,25 puntos.
  - Extensión: 0,75 puntos.
- La fecha límite de entrega es el 11 de diciembre de 2026 a las 23:59.

2. El conjunto de datos

El conjunto de datos se proporciona en el fichero Excel projects.xlsx. Contiene 35.378 registros correspondientes a proyectos financiados por la Comisión Europea dentro de H2020. Para cada proyecto se proporciona la siguiente información:

- projectID: identificador del proyecto.
- acronym: acrónimo del proyecto.
- title: título del proyecto.
- summary: resumen del proyecto.
- startDate, endDate: fechas de inicio y fin de ejecución.
- totalCost: coste total del proyecto.
- ecMaxContribution: financiación de la UE.
- countryContr: financiación recibida por cada país.
- coordinatorCountry: país coordinador del proyecto.
- euroSciVocCode: código de la categoría temática del proyecto. Las categorías asociadas a cada código pueden consultarse en el fichero SciVocCodes.xlsx.
- publicationID: identificadores de las publicaciones asociadas al proyecto.
- patentID: identificadores de las patentes asociadas al proyecto.

La base de datos se ha construido a partir de la información publicada por CORDIS en el portal de datos de la Unión Europea: https://data.europa.eu/data/datasets/cordish2020projects

3. Definición del problema de clasificación multiclase

Las categorías temáticas proceden de EuroSciVoc, el Vocabulario Europeo de Ciencia. Se trata de una jerarquía de más de mil conceptos, con hasta seis niveles, que cuelgan de siete raíces que

siguen los dos primeros niveles de la clasificación de Campos de Investigación y Desarrollo de la OCDE.

Un proyecto puede tener asociados varios términos EuroSciVoc, y cada término constituye un camino dentro de esa jerarquía.

Esto deja abiertas dos decisiones: qué nivel de la jerarquía predecir y qué hacer con los proyectos que tienen varios códigos. Para que todos los equipos resuelvan el mismo problema y los resultados sean comparables, el proyecto básico fija ambas decisiones:

- Objetivo: el segundo nivel de la jerarquía. Para cada término EuroSciVoc de cada proyecto, recorred la jerarquía hacia arriba hasta encontrar su ancestro de nivel 2. Los códigos que no alcancen el segundo nivel deben descartarse.
- Una etiqueta por proyecto. Para cada proyecto, utilizad la primera categoría EuroSciVoc disponible para la que pueda extraerse un objetivo válido de segundo nivel. La versión multietiqueta del problema queda como una de las extensiones propuestas y no forma parte del proyecto básico.

Después de aplicar los dos criterios anteriores, deben descartarse los proyectos para los que no haya sido posible obtener una categoría válida de segundo nivel. Como punto de control, podéis comprobar que se conserva aproximadamente el 90,31 % de los proyectos proporcionados. Además de la categoría de segundo nivel, los proyectos disponen también de su categoría correspondiente de primer nivel. Esta información de primer nivel, con solo seis clases (una de las siete raíces del vocabulario no es utilizada por ningún proyecto de este conjunto de datos), se utilizará como control; véase la Sección 4.4.

Una advertencia importante sobre las etiquetas de los proyectos. Los términos EuroSciVoc de cada proyecto fueron sugeridos mediante herramientas automáticas de procesamiento del lenguaje aplicadas a las descripciones de los proyectos y posteriormente validados por expertos humanos. Es decir, parte del etiquetado de referencia (ground truth) que vais a predecir fue generado a partir del mismo texto que utilizaréis como entrada. Esto no invalida el ejercicio —las etiquetas han sido revisadas y son las que realmente utiliza CORDIS—, pero sí significa que una puntuación muy alta debería haceros sospechar que parte del éxito de la clasificación puede deberse a la forma en que se generaron las etiquetas disponibles.

El coste de la simplificación. Convertir un problema multietiqueta en uno de etiqueta única es una decisión de modelado y afecta a la interpretación de los resultados. En este conjunto de datos, solo alrededor del 32 % de los proyectos tienen todos sus términos EuroSciVoc bajo la misma categoría de segundo nivel; los dos tercios restantes abarcan dos o más áreas, y la etiqueta que predeciréis es simplemente la primera de ellas. Si en lugar de «el primer término» se utilizara «el ancestro de segundo nivel más frecuente entre los términos del proyecto», la etiqueta cambiaría en aproximadamente el 12 % de los proyectos. En el primer nivel de la jerarquía la situación es similar: solo el 47 % de los proyectos son inequívocos y, de nuevo, aproximadamente el 12 % de las etiquetas cambiarían si se utilizara la regla de la mayoría.

A continuación se indican dos consecuencias que podéis incorporar a vuestra discusión:

- Existe un techo. Una parte considerable de cualquier desacuerdo entre vuestro clasificador y la etiqueta se debe a la arbitrariedad de escoger una de las varias áreas a las que el proyecto pertenece realmente. No esperéis ni persigáis tasas de clasificación cercanas al 100 %.
- Las confusiones son informativas. Al analizar dónde fallan los modelos (Sección 4.4), podéis medir con qué frecuencia la clase predicha es una de las categorías de segundo nivel descartadas. Esta medida puede ayudar a distinguir cuánto del error de clasificación se debe a la ambigüedad de la etiqueta y cuánto a un fallo real del modelo. Es uno de los valores más interesantes que podéis reportar.

4. Proyecto básico

Para obtener la calificación mínima de aprobado, vuestro trabajo debe incluir todos los apartados siguientes.

4.1. Análisis exploratorio

Antes de aplicar cualquier técnica de procesamiento del lenguaje, realizad un breve análisis exploratorio del conjunto de datos:

- Descripción general (número de instancias, variables, tipos de datos y valores ausentes).
- Distribución de la variable objetivo una vez construida como se describe en la Sección 3. ¿Cuántas clases quedan y cuál es su grado de desbalance? ¿Qué rendimiento obtendría un clasificador que siempre predijera la clase mayoritaria? Ese valor es el modelo de referencia (baseline) que todo lo demás debe superar.
- Estadísticas básicas y visualizaciones sencillas (longitud de los resúmenes, proyectos por año y por país).
- Análisis preliminar del texto (términos más frecuentes, nube de palabras y ejemplos por clase).

4.2. Representación vectorial

Construid tres representaciones diferentes de los documentos:

- TF-IDF: representación basada en la frecuencia de término y la frecuencia inversa de documento.
- Word2Vec: representación de cada documento como la media de los embeddings de las palabras que contiene. Indicad si entrenáis los embeddings con este corpus o si utilizáis vectores preentrenados, y cómo tratáis las palabras fuera de vocabulario (OOV).
- Embeddings contextuales obtenidos a partir de modelos Transformer preentrenados de tipo encoder-only, como BERT o RoBERTa. Explicad cómo convertís la salida a nivel de token en un único vector por documento y qué hacéis con los resúmenes de proyecto que superan la longitud máxima de entrada del modelo.

Para calcular la representación TF-IDF será necesario implementar un pipeline de preprocesamiento del texto. Podéis utilizar las bibliotecas habituales (NLTK, Gensim, SpaCy) o cualquier otra que consideréis adecuada. Documentad las decisiones adoptadas (por ejemplo, conversión a minúsculas, palabras vacías (stopwords), lematización y frecuencias mínima y máxima de documento). El uso del pipeline de preprocesamiento para Word2Vec es opcional si decidís entrenar vosotros mismos los embeddings. Para los embeddings contextuales debe utilizarse únicamente el texto sin preprocesar.

El Apéndice A ofrece una descripción concisa de las representaciones TF-IDF y Word2Vec, incluyendo el preprocesamiento de texto. Contiene los conceptos básicos necesarios y algunos consejos de cálculo para que podáis empezar a trabajar en el proyecto desde el principio.

4.3. Modelado

Entrenad, ajustad y evaluad modelos de clasificación utilizando, como mínimo, las siguientes estrategias:

- Al menos un algoritmo de Scikit-learn (por ejemplo, k-NN, SVM, random forest o regresión logística).
- Una red neuronal diseñada e implementada en PyTorch.

La selección de hiperparámetros debe realizarse utilizando un procedimiento de validación adecuado, sin utilizar el conjunto de prueba (test) para tomar decisiones de modelado. El informe debe indicar claramente el tamaño de los conjuntos de entrenamiento, validación y prueba, y el código debe permitir reproducir los experimentos.

Además, comparad estos modelos con un Transformer preentrenado adaptado a la tarea mediante fine-tuning con la biblioteca Hugging Face Transformers. Es importante distinguir este experimento del uso anterior de embeddings contextuales: en aquel caso el Transformer actúa como extractor de características y sus pesos permanecen congelados; durante el fine-tuning, en cambio, los parámetros del Transformer se actualizan utilizando las etiquetas del problema de clasificación. El fine-tuning de un modelo base sobre unas decenas de miles de resúmenes puede realizarse en un tiempo razonable con una GPU gratuita de Google Colab; no se espera que lo ejecutéis en vuestro ordenador personal.

4.4. Evaluación comparativa

Se espera que:

- Evaluéis los modelos con métricas adecuadas para un problema de clasificación multiclase con clases desbalanceadas, seleccionadas y justificadas. La exactitud (accuracy) por sí sola no es suficiente; deben calcularse y utilizarse en la discusión el F1 macro (macro-F1) y el desglose por clase.
- Comparéis el rendimiento obtenido con las distintas representaciones de los documentos y con los diferentes algoritmos de aprendizaje.
- Repitáis vuestra mejor configuración utilizando el primer nivel de la jerarquía (seis clases) y comparéis los resultados. ¿Cambia el ranking de las tres representaciones al variar la granularidad del problema?
- No interpretéis diferencias que estén dentro de la incertidumbre de vuestro conjunto de prueba (test). Indicad qué magnitud debería tener una diferencia para que fuese significativa y aplicad ese criterio a vuestras propias conclusiones.
- Analicéis dónde fallan los modelos. ¿Las confusiones se producen entre categorías hermanas de la taxonomía, lo cual sería razonable, o entre categorías no relacionadas?

El objetivo del informe no es únicamente presentar métricas, sino justificar las decisiones de modelado y analizar críticamente los resultados. Apoyad las conclusiones con tablas, figuras y análisis de errores que permitan entender las diferencias entre representaciones y modelos.

5. Extensión

La extensión es libre: ampliad el proyecto básico en la dirección que consideréis más apropiada. A continuación se proponen algunas ideas, aproximadamente en orden creciente de ambición:

- **¿Qué aporta el texto?** El conjunto de datos contiene metadatos que no tienen relación directa con el resumen: coste, contribución de la UE, duración, país coordinador y número de países participantes. Construid un clasificador que utilice solo esos datos, otro que utilice únicamente texto y un tercero que combine ambos. ¿Cuánto de lo conseguido por vuestro modelo de texto podría haberse obtenido sin utilizar los resúmenes de los proyectos?
- **Clasificación zero-shot con un LLM.** Todos los modelos del proyecto básico aprenden a partir del conjunto de entrenamiento. Un LLM capaz de seguir instrucciones no lo necesita: dado el resumen y la lista de categorías, puede responder sin haber visto ningún ejemplo etiquetado. Comparad los resultados del proyecto básico con una estrategia zero-shot mediante prompts utilizando modelos servidos localmente como Qwen, Mistral o Gemma.
- **Clasificación few-shot.** Una continuación natural consiste en añadir unos pocos ejemplos al prompt y plantear la pregunta en sentido inverso: ¿cuántos ejemplos etiquetados necesita vuestro pipeline supervisado antes de superar a un modelo al que no se le ha proporcionado ninguno?

Como punto de comparación más económico, el pipeline zero-shot basado en NLI de Hugging Face (zero-shot-classification) resuelve la misma tarea con un modelo mucho más pequeño.

- **Clasificación multietiqueta.** Recuperad las etiquetas descartadas en la Sección 3 y tratad el problema tal y como es realmente: cada proyecto puede pertenecer a varias categorías. Esto cambia la capa de salida, la función de pérdida y, sobre todo, las métricas: subset accuracy, F1 micro y macro, y la decisión de dónde situar el umbral.
- **¿Ayuda la jerarquía?** Comparad un clasificador plano sobre las categorías de segundo nivel con uno jerárquico que primero prediga la raíz y después la subcategoría condicionada a ella. Hay argumentos en ambas direcciones; resolvedlo con vuestros datos.
- **¿La estructura no supervisada recupera la taxonomía? Aplicad modelado de temas (topic modeling) o clustering sobre los embeddings contextuales, sin utilizar las etiquetas, y comparad los grupos obtenidos con EuroSciVoc mediante una medida de acuerdo como el índice de Rand ajustado (Adjusted Rand Index) o la información mutua normalizada (Normalized Mutual Information). Analizar dónde discrepan la taxonomía no supervisada y los datos etiquetados resulta más interesante que analizar dónde coinciden.**
- **Predicción de producción científica.** Formulad una tarea de regresión sobre el número de publicaciones o patentes asociadas. Tened en cuenta que es un problema bastante más difícil de lo que parece y que el tamaño y la duración del proyecto explican una parte importante de la varianza: la pregunta interesante vuelve a ser qué aporta el resumen además de esas variables.
- **Comparación de embeddings contextuales.** Comparad varios modelos Transformer, incluidos modelos específicos del dominio (SciBERT y similares), y estudiad el efecto de esa elección sobre el rendimiento final.

Para la opción de clasificación zero-shot con un LLM, podéis abordar en el informe los siguientes aspectos:

- El modelo puede responder fuera de la taxonomía. Con varias decenas de categorías en el prompt, el modelo puede inventar, fusionar o parafrasear etiquetas. Decidid cómo mapear una respuesta a una categoría válida e informad de la fracción de respuestas que necesitaron ser corregidas o rescatadas: ese valor es un resultado, no un error de implementación.
- La inferencia es lenta. No podréis etiquetar 35.000 resúmenes en un portátil. Tomad una submuestra del conjunto de prueba (test), indicad su tamaño y evaluad los modelos supervisados exactamente sobre ese mismo subconjunto para que la comparación tenga sentido.
- La granularidad afecta aún más. Ejecutad la estrategia zero-shot en los niveles 1 y 2 y comparad la degradación con la observada en los modelos supervisados.

Podéis servir localmente modelos cuantizados de 7–8B mediante Ollama o Transformers, o solicitar al profesor acceso a un servicio basado en Ollama proporcionado por el departamento TSC-UC3M vía HTTP.

Tomad esta lista únicamente como sugerencia; podéis elegir cualquier otro tema siempre que encaje dentro del ámbito de la asignatura. Se valorará la originalidad de la elección. Si tenéis dudas sobre la idoneidad de la extensión elegida, consultad al profesor. En cualquier caso, recordad que la extensión supone 0,75 puntos de la nota final: evitad extensiones excesivamente ambiciosas que puedan comprometer la entrega del proyecto en plazo.

6. Entrega del proyecto

La entrega utilizará GitHub como herramienta de colaboración y control de versiones, y deberá incluir:

- Código completamente funcional y documentado.
- Un fichero README.md (o un sitio de GitHub Pages) que actúe como informe del proyecto. No es una guía de instalación: debe contener tanto el informe de la implementación como el análisis de los resultados, incluyendo:
  - Una descripción del problema y del conjunto de datos.
  - Una explicación de las metodologías utilizadas, con suficiente detalle para que el lector sepa qué se ejecutó realmente: decisiones de preprocesamiento, representaciones, modelos, búsqueda de hiperparámetros y protocolo de entrenamiento/validación/test.
  - Resultados experimentales y discusión. Las gráficas y tablas en las que se apoye la discusión deben estar incluidas en el propio README, no únicamente dentro de los notebooks: curvas de aprendizaje o validación, comparación entre representaciones y modelos, matriz de confusión de la mejor configuración y cualquier otro resultado utilizado en la argumentación. Toda afirmación del análisis debe estar respaldada por una figura o tabla que el lector pueda consultar sin ejecutar el código.
  - Conclusiones, incluyendo las limitaciones de las que seáis conscientes.
- Todos los ficheros o scripts necesarios para reproducir los resultados. No subáis projects.xlsx al repositorio; incluid en su lugar el código que lea los ficheros de datos desde una ruta local.

El repositorio de GitHub debe ser público para que los profesores puedan revisar y reproducir el trabajo. Las páginas de documentación no podrán editarse después de la fecha límite; la evaluación se basará en el último commit realizado antes de dicha fecha.

La entrega se realizará a través de Aula Global. Solo un estudiante de cada grupo deberá subir:

- El enlace al repositorio de GitHub.
- Una breve declaración de contribuciones indicando qué hizo cada miembro del equipo.
- Las diapositivas que utilizaréis en la presentación del proyecto. Se permiten pequeños cambios en las diapositivas antes de la presentación final.

Sin excepción, la documentación debe respetar el principio de reconocimiento adecuado de autoría. Si utilizáis fragmentos de código externos o cualquier material procedente de otras fuentes, debéis indicarlo claramente en el informe.

7. Evaluación

El proyecto se evaluará a partir de la documentación entregada y de una presentación individual que tendrá lugar durante la semana del 14 de diciembre de 2026. La calidad metodológica del trabajo se evaluará en ambos lugares: en la documentación y en la forma en que defendáis vuestras decisiones cuando se os formulen preguntas.

Criterio	Puntuación
Proyecto básico	2,25 puntos
Metodología	0,75
Calidad de la documentación	0,30
Calidad del código	0,30
Calidad de la presentación	0,50
Respuestas a los comentarios/preguntas de la presentación	0,40
Extensión	0,75 puntos
Originalidad	0,25
Calidad del trabajo	0,50

Qué no se evalúa: la exactitud (accuracy) alcanzada. Ninguno de los criterios de la tabla anterior recompensa una rendimiento de clasificación alta. El conjunto de datos, la etiqueta y el techo discutido en la Sección 3 son los mismos para todos los equipos, y una diferencia de unos pocos puntos en macro-F1 suele decir más sobre la arbitrariedad de la etiqueta que sobre la calidad del trabajo realizado. Lo que se evalúa es la corrección metodológica —un protocolo limpio de entrenamiento/validación/test, hiperparámetros ajustados sin utilizar el conjunto de prueba (test), métricas adecuadas para un problema de clasificación multiclase con clases desbalanceadas y comparaciones realmente comparables— y la discusión crítica de los resultados obtenidos, sean cuales sean. Un resultado modesto que hayáis medido correctamente, comprendáis y sepáis explicar vale más que una cifra mejor que no podáis justificar. Informar de un resultado negativo —una representación que no ayuda, una extensión que no supera el modelo de referencia (baseline)— es un resultado perfectamente válido, siempre que vuestro análisis explique por qué.

Esto se aplica en los mismos términos al proyecto básico y a la extensión, y en los dos lugares en los que se evalúa el trabajo: el README y la presentación. En particular, no dediquéis la presentación a enumerar cifras; centradla en qué hicisteis, por qué lo hicisteis y qué conclusiones obtuvisteis de los resultados.

Apéndice A. Representaciones de entrada: preprocesamiento, TF-IDF y Word2Vec

Este apéndice es un resumen práctico de lo necesario para construir las representaciones TF-IDF y Word2Vec antes de que estos temas se cubran en las clases teóricas. Es deliberadamente breve y práctico, y no sustituye a las clases: cuando lleguemos a estos contenidos entenderéis por qué funcionan estas representaciones.

A.1. Preprocesamiento de texto

TF-IDF trata un documento como una bolsa de términos, por lo que la primera decisión es qué se considera un término. Un pipeline estándar aplica, en este orden:

1. Tokenización: dividir el texto en tokens (palabras). Una expresión regular sobre caracteres de palabra puede ser suficiente; SpaCy o NLTK gestionan mejor guiones, acrónimos y unidades.
2. Normalización de mayúsculas/minúsculas: convertir todo a minúsculas. Se pierde la distinción entre US y us, pero se obtiene un vocabulario mucho más pequeño.
3. Eliminación de tokens no informativos: puntuación, números, tokens de uno o dos caracteres y palabras vacías (stopwords) («the», «of», «which»), para las que todas las bibliotecas proporcionan una lista estándar en inglés.
4. Reducción de flexiones: mapear networks, networking y network a un único término. El stemming (Porter, Snowball) elimina sufijos, es rápido y produce formas que pueden no ser palabras reales; la lematización (SpaCy, WordNet) devuelve formas de diccionario reales y es más lenta. Cualquiera de las dos opciones es aceptable; indicad cuál utilizasteis.
5. Poda del vocabulario por frecuencia de documento: eliminar términos que aparecen en menos de n documentos (ruido, erratas, nombres aislados) y términos que aparecen en casi todos los documentos (no aportan información discriminativa).

Un pipeline mínimo con SpaCy, aplicado a la columna summary:

```
import spacy
nlp = spacy.load("en_core_web_sm", disable=["parser", "ner"])

def clean(doc):
    return [t.lemma_.lower() for t in doc
```

```
            if t.is_alpha and not t.is_stop and len(t) > 2]

docs = [clean(d) for d in nlp.pipe(summaries, batch_size=256, n_process=4)]
```

Una observación específica de este corpus: los resúmenes de proyectos contienen muchas expresiones repetitivas («this project», «work package», «state of the art», «consortium»). Estas expresiones se comportan como palabras vacías (stopwords) específicas del dominio, y la poda mediante frecuencia máxima de documento permite eliminarlas sin necesidad de enumerarlas manualmente.

A.2. TF-IDF

Sea D un corpus de N documentos y V el vocabulario obtenido tras el preprocesamiento. El peso del término t en el documento d es el producto de dos factores:

$$\text{tfidf}(t,d) = \text{tf}(t,d) \cdot \text{idf}(t), \qquad \text{idf}(t) = \log(N / \text{df}(t))$$

donde tf(t,d) es el número de apariciones de t en d y df(t) es el número de documentos que contienen t. El primer factor indica que un término repetido en un documento es importante para ese documento; el segundo indica que un término que aparece en todas partes no discrimina. Después, cada vector de documento se normaliza a norma ℓ2 unitaria, de modo que los resúmenes largos y cortos sean comparables. Existen varias variantes: scikit-learn utiliza por defecto una versión suavizada del IDF y normalización ℓ2.

El resultado es una matriz N × |V| en la que menos del 1 % de las entradas son distintas de cero. Mantenedla en formato disperso (scipy.sparse).

```
from sklearn.feature_extraction.text import TfidfVectorizer

vec = TfidfVectorizer(min_df=5,          # descarta términos en < 5 documentos
                      max_df=0.5,        # descarta términos en > 50% de documentos
                      ngram_range=(1,1), # (1,2) añade también bigramas
                      sublinear_tf=True)
X_tr = vec.fit_transform(docs_tr)        # ajustar SOLO con conjunto de entrenamiento
X_te = vec.transform(docs_te)
```

Los hiperparámetros más relevantes son min_df y max_df (controlan |V| y, por tanto, todo el problema), ngram_range (los bigramas capturan expresiones como «machine learning» y «climate change», a costa de multiplicar el vocabulario) y max_features. Si preferís trabajar con Gensim, los objetos equivalentes son Dictionary (con filter_extremes) y TfidfModel.

Una nota práctica para la sección de modelado: los modelos lineales (regresión logística, SVM lineal) son muy eficaces con TF-IDF disperso y de alta dimensionalidad, mientras que k-NN y los ensembles de árboles tienden a tener dificultades. Si queréis utilizarlos, reducid primero la dimensionalidad.

A.3. Word2Vec

Word2Vec representa cada palabra como un vector denso de dimensión d (normalmente entre 100 y 300), aprendido a partir de un corpus grande de forma que las palabras que aparecen en contextos similares obtengan vectores similares. Existen dos objetivos de entrenamiento —CBOW, que predice una palabra a partir de su contexto, y skip-gram, que predice el contexto a partir de la palabra—, pero no necesitáis conocer sus detalles para utilizar la representación. Lo importante es el contraste con TF-IDF: las dimensiones no son términos, no tienen significado individual, los vectores son densos y palabras semánticamente relacionadas (photovoltaic, solar) quedan próximas aunque sean términos diferentes.

Tenéis dos opciones, y no son equivalentes:

- Entrenar los embeddings con este corpus. El vocabulario coincide exactamente con el vuestro, los términos técnicos están cubiertos y no hay palabras fuera de vocabulario (OOV); sin embargo, 35.000 resúmenes constituyen un corpus pequeño para estimar buenos embeddings.
- Utilizar vectores preentrenados (GloVe, fastText, word2vec-google-news-300, todos disponibles mediante gensim.downloader). Están mucho mejor estimados a partir de miles de millones de palabras, pero faltarán términos específicos del dominio y tendréis que decidir cómo tratarlos.

Word2Vec proporciona vectores de palabras, pero necesitáis un único vector por documento. El procedimiento estándar consiste en promediar los vectores de las palabras del documento, opcionalmente ponderándolos mediante TF-IDF:

```
import numpy as np
from gensim.models import Word2Vec

w2v = Word2Vec(docs_tr, vector_size=200, window=5, min_count=5, sg=1, workers=4)

def doc_vector(tokens, kv, dim):
    vs = [kv[t] for t in tokens if t in kv]        # omite palabras OOV
    return np.mean(vs, axis=0) if vs else np.zeros(dim)

Z = np.vstack([doc_vector(d, w2v.wv, 200) for d in docs])
```

Indicad cómo habéis tratado las palabras fuera de vocabulario (OOV) y con qué frecuencia un documento termina sin ningún token utilizable. Tened en cuenta también que el promedio destruye el orden de las palabras y diluye el contenido de documentos largos, precisamente una de las razones por las que existen los embeddings contextuales. Con resúmenes de 100–200 palabras, el promedio funciona razonablemente bien; no ocurriría lo mismo con tweets. La matriz resultante es densa y sus características tienen escalas muy diferentes, por lo que debe estandarizarse (StandardScaler) antes de utilizar modelos basados en distancias o regularizados.

A.4. Una advertencia aplicable a las tres representaciones

El vocabulario, los pesos IDF, cualquier embedding que entrenéis con el corpus y cualquier método de escalado (scaler) son parámetros estimados a partir de los datos. Ajustadlos únicamente con la partición de entrenamiento y aplicadlos después a validación y test. Ejecutar el vectorizador sobre todo el conjunto de datos antes de realizar la partición es el error metodológico más frecuente en este proyecto y produce resultados artificialmente inflados. Utilizar un sklearn.pipeline.Pipeline dentro de la validación cruzada hace que sea mucho más difícil cometer este error.